
UNIT 9 EXPERT SYSTEMS

Structure	Page Nos.
9.0 Introduction	277
9.1 Objectives	277
9.2 An Introduction to Expert Systems	278
9.3 Concept of Planning, Representing, and Using Domain Knowledge	279
9.4 Knowledge Representation Schemes	279
9.4 Examples of Expert Systems: MYCIN, COMPASS	286
9.6 Expert System Building Tools	287
9.6.1 Expert System Shells	288
9.6.1.1 Knowledge Base	289
9.6.1.2 Knowledge Acquisition Subsystem (Example: COMPASS)	289
9.6.1.3 Inference Engine	290
9.6.1.4 Explanation Sub-system (Example: MYCIN)	290
9.6.1.5 User Interface	293
9.6.1.6 An Expert System shell: EMYCIN	293
9.7 Some Application Areas of Expert Systems	294
9.8 Summary	294
9.9 Solutions/Answers	295
9.10 Further Readings	295

9.0 INTRODUCTION

Computer Science is the study of creating models that can be represented and executed by computing systems. New types of problems are being addressed almost every day through the development of models that represent the domains of the problems under consideration. One of the major problems facing humanity is the scarcity of human experts capable of addressing problems across various domains of human experience, including health, education, economic welfare, natural resources, and the environment. In this respect, the task of a computer scientist is to create, in addition to a model of the problem domain, a model of a domain expert as a problem solver who is highly skilled in addressing problems within that domain. The field of expert systems is concerned with creating such models. The task includes activities related to eliciting information from the experts in the domain with respect to how they solve the problems, and activities related to codifying that information generally in the form of rules. This unit discusses such issues about the design and development of an expert system.

9.1 OBJECTIVES

After going through this unit, you should be able to:

- tell about some of the well-known expert systems;
- enumerate and use various tools for building expert systems;
- explain the various expert system shells, and
- discuss various application areas for expert systems.

First of all, we must understand that an expert system is nothing but a computer program or a set of computer programs that contains knowledge and some inferential capability of an expert, most generally a human expert, in a particular domain. Since an expert system is designed to draw conclusions based on the inputs provided, the information it already contains, and its processing capabilities, it belongs to the branch of Computer Science known as Artificial Intelligence.

Mere possessing an algorithm for solving a problem is not sufficient for a program to be termed an expert system; **it must also possess knowledge**, i.e., if there is an expert system for a particular domain or area, and if it is fed with a number of questions regarding that domain, then sooner or later, we can expect that these questions will be answered. So, we can say that the knowledge contained by an expert system must contribute towards solving the problems for which it has been designed.

Also, knowledge in an expert system must be regarding a **specific domain**. As a human being cannot be an expert in every area of life, similarly, an expert system, which tries to simulate the capabilities of an expert, also works in a particular domain. Otherwise, it may be required to possess an infinite amount of knowledge, and processing that knowledge in a finite amount of time is an impossible task.

Taking into consideration all the points which have been discussed above, let us try to give one of the many possible definitions of an Expert System:

An Expert System is a computer program that possesses or represents knowledge in a particular domain, has the capability of processing/manipulating or reasoning with this knowledge with a view to solving a problem, or to achieve some specific goal.

An expert system may or may not provide the complete expertise or functionality of a human expert, but it must be able to assist a human expert in fast decision making. The program might interact with a human expert or with a customer directly.

Let us discuss some of the basic properties of an expert system:

- It tries to simulate human reasoning capability about a specific domain rather than the domain itself. This feature separates expert systems from some other familiar programs that use mathematical modeling or computer animation. In an expert system, the focus is to emulate an expert's knowledge and problem-solving capabilities, and if possible, at a faster rate than a human expert.
- It performs reasoning over the acquired knowledge, rather than merely performing some calculations or performing data retrieval.
- It can solve problems by using heuristic or approximate models, which, unlike other algorithmic solutions, are not guaranteed to succeed.

AI programs that achieve expert-level competence in solving problems in different domains are often called **knowledge-based systems**. A **knowledge-based system** is any system that performs a task by applying rules of thumb to a symbolic representation of knowledge, rather than relying primarily on algorithmic or statistical methods. Often, the term *expert system* is reserved for programs whose knowledge base contains the knowledge acquired from human experts, as opposed to knowledge obtained from textbooks or non-experts. However, more often than not, the terms expert systems and knowledge-based systems are used interchangeably.

Together they represent the most widespread type of *AI* application. The area of human intellectual endeavour that is to be captured in an expert system is sometimes called the task domain. A **task** refers to a goal-oriented, problem-solving activity, while a **domain** refers to the area within which the task is performed. Typical tasks include diagnosis, planning, scheduling, configuration, and design. For example, a program capable of conversing about the weather would be a knowledge-based system, even if that program did not have any expertise in meteorology. An expert system, however, must be capable of performing tasks such as weather forecasting, which require domain-specific expertise.

Building an expert system is known as **knowledge engineering**, and its practitioners are called **knowledge engineers**. It is the job of the knowledge engineer to ensure that the computer has all the knowledge it needs to solve a problem. The knowledge engineer must choose one or more appropriate forms for representing the required knowledge, i.e., select one or more knowledge representation schemes (several knowledge representation schemes, including semantic nets, frames, predicate logic, and rule-based systems, have been discussed above). We would be sticking to a rule-based representation scheme for our discussion on expert systems. S/he must also ensure that the computer can use the knowledge efficiently by selecting from a handful of reasoning methods.

9.3 CONCEPT OF PLANNING, REPRESENTING AND USING DOMAIN KNOWLEDGE

From our everyday experience, we know that in order to solve difficult problems, we need to do some sort of planning. Informally, we can say that **planning** is the process that exploits the structure of the problem under consideration for designing a sequence of actions in order to solve the problem under consideration.

The knowledge of the nature and structure of the problem domain is essential for planning a solution to the problem under consideration. For the purpose of planning, the problem environments are divided into two categories, viz., classical planning environments and non-classical planning environments. The **classical** planning environments/domains are fully observable, deterministic, finite, static, and discrete. On the other hand, **non-classical** planning environments may be only partially observable and/or stochastic. In this unit, we discuss planning only for classical environments.

9.4 KNOWLEDGE REPRESENTATION SCHEMES

One of the underlying assumptions in Artificial Intelligence is that **intelligent behaviour can be achieved through the manipulation of symbol structures** (representing bits of knowledge). One of the main issues in *AI* is to find an appropriate representation of problem elements and available actions as symbol structures so that the representation can be used to intelligently solve problems. In *AI*, **an important criterion for knowledge representation schemes or languages is that they should support inference**. For intelligent action, the inferencing capability is essential in view of the fact that we can't represent explicitly everything that the system might ever need to know—**some things have to be left implicit, to be inferred/deduced by the system** as and when needed in problem solving.

In general, a good knowledge representation scheme should have the following features:

- It should allow us to express the knowledge we wish to represent in the language. For example, the mathematical statement: *Every symmetric and transitive relation on a domain need not be reflexive* is not expressible in First Order Logic.
- It should allow new knowledge to be inferred from a basic set of facts, as discussed above.
- It should have well-defined syntax and semantics.

Some popular knowledge representation schemes are:

- **Semantic networks,**
- **Frames,**
- **First order logic, and,**
- **Rule-based systems.**

The above knowledge representation schemes have already been discussed in the previous unit (Please refer to Unit 6).

Proposition and Predicate Logic

Symbolic logic may be thought of as a formal language that incorporates a precise system for reasoning and deduction. Propositional logic and predicate logic are two well-known forms of symbolic logic. Propositional logic deals with propositions or statements.

A proposition or a statement is a sentence that can be assigned a truth-value, *true* or *false*. For example, the statement:

The sun rises in the west, has a truth value *false*.

On the other hand, none of the following sentences can be assigned a truth-value, and hence none of these is a statement or proposition:

- (i) *Ram must exercise regularly.* (Imperative sentence)
- (ii) *Who was the first Prime Minister of India?* (Interrogative sentence)
- (iii) *Please, give me that book.* (Imperative sentence)
- (iv) *Hurry! We have won the trophy.* (Exclamatory sentence).

Generally, the following **steps are followed for solving problems using propositional logic**, where the problems are expressed in English and are such that they can be solved using propositional logic (**PL**):

- a) First problem statements in English are translated to formulae of Propositional logic

- b) Next, some of the rules of inference in PL, including the ones mentioned below, are used to solve the problem, if, at all, the problem under consideration is solvable by PL.

Some of the rules of Inference of Propositional Logic:

$$\begin{array}{l} \text{(i) Modus Ponens} \quad P \\ \quad \quad \quad P \rightarrow Q \\ \hline \quad \quad \quad Q \end{array}$$

(The above notation is interpreted as: if we are given the two formulae P and $P \rightarrow Q$ of the propositional logic, then conclude the formula Q . In other words, if both the formulae P and $P \rightarrow Q$ are assumed to be true, then by modus ponens, we can assume the statement Q also as true.)

$$\begin{array}{l} \text{(ii) Chain Rule} \\ \\ P \rightarrow Q \\ Q \rightarrow R \\ \hline P \rightarrow R \end{array}$$

$$\begin{array}{l} \text{(iii) Transposition} \\ \\ P \rightarrow Q \\ \hline \sim Q \rightarrow \sim P \end{array}$$

Example

Given the following three statements:

- (i) Matter always existed
- (ii) If there is God, then God created the universe
- (iii) If God created the universe, then matter did not always exist.

To show the truth of the statement: *There is no God.*

Solution:

Let us denote the atomic statements in the argument given above as follows:

M: Matter always existed

TG: There is God

GU: God created the universe.

Then **the given** statements in English become the formulae of PL, respectively:

- (i) M
- (ii) $TG \rightarrow GU$
- (iii) $GU \rightarrow \sim M$
- (iv) **To show** $\sim TG$

Applying transposition to (iii), we get

- (v) $M \rightarrow \sim GU$

using (i) and (v) and applying Modus Ponens, we get

- (vi) $\sim GU$

Again, applying transposition to (ii), we get

- (vii) $\sim GU \rightarrow \sim TG$

Applying Modus Ponens to (vi) and (vii), we get

- (viii) $\sim TG$

The formula (viii) is the same as formula (iv), which was required to be proved.

Exercise 1: Using propositional logic, show that, if the following statements are assumed to be true:

- (i) *There is a moral law.*
- (ii) *If there is a moral law, then someone gave it.*

(iii) If someone gave the moral law, then there is God.

then the following statement is also true:

(iv) There is God.

The trouble with propositional logic is that it is unable to describe properties of objects, and it also lacks the structure to express relations that exist among two or more entities. Further, propositional logic does not allow us to make generalised statements about classes of similar objects. However, in many situations, the explicit knowledge of relations between objects and generalised statements about classes of similar objects is essentially required to solve problems. Thus, propositional logic has serious limitations while reasoning for solving real-world problems. For example, let us look at the following statements:

(i) All children more than 12 years old must exercise regularly.

(ii) Ram is more than 12 years old.

(iii) Now, these statements should be sufficient enough to allow us to conclude: *Ram must exercise regularly*. However, in propositional logic, each of the above statements is indecomposable and may be respectively denoted by P and Q. Further, whatever is said inside each statement is presumed to be not visible. Therefore, if we use the language of propositional logic, we are just given two symbols, viz., P and Q, representing the two given statements respectively. However, from just two propositional formulae, P and Q, it is not possible to conclude the above-mentioned statement, viz., *Ram must exercise regularly*. To draw the desired conclusion with a valid inference rule, it would be necessary to use some other language, including some extension of propositional logic.

(iv)

(v) *Predicate Logic*, and more specifically, **First Order Predicate Logic (FOPL)** is an extension of propositional logic developed to enhance its expressive power. In addition to the propositions used in propositional logic, predicate logic uses predicates, functions, and variables together with variable quantifiers (*Universal and Existential quantifiers*) to express knowledge.

(vi) We have already defined the structure of formulae of FOPL and have also explained the procedure for finding the meaning of formulae in FOPL. Though, we have already explained how to solve problems using FOPL, yet just for recalling the procedure for solving problems using FOPL, we will consider one example as given below.

(vii) In this context, we may recall the **inference rules of FOPL**. The inference rules of PL, including *Modus Ponens*, *Chain Rule* and *Rule of Transposition*, are valid in FOPL also after suitable modifications by which formulae of PL are replaced by formulae of FOPL.

In addition to these inference rules, the **following four inference rules of FOPL**, which will be called Q₁, Q₂, Q₃, and Q₄, have no corresponding rules in PL. In the following, F denotes a predicate and x a variable/parameter:

$$Q_1: \frac{\sim (\exists x)F(x)}{(\forall x) \sim F(x)} \quad \text{and} \quad \frac{(\forall x) \sim F(x)}{\sim (\exists x)F(x)}$$

The first of the above rules under Q₁, says:

From *negation of there exists x F (x)*, we can infer *for all x not of F (x)*

$$\frac{\sim (\forall x)F(x)}{(\exists x) \sim F(x)} \quad \text{and} \quad \frac{(\exists x) \sim F(x)}{\sim (\forall x)F(x)}$$

The first of the above rules under Q₂, says:

From *negation of for all x F(x)*, we can infer that *there exists x such that not of F(x)*

$$Q_3: \frac{(\forall x)F(x)}{F(a)}, \text{ where } a \text{ is (any) arbitrary element of the domain of } F$$

The rule Q₃ is called **universal instantiation**

$$Q_3: \frac{F(a), \text{ for arbitrary } a}{(\forall x)F(x)}$$

The rule is called **universal generalisation**

$$Q_4: \frac{(\exists x)F(x)}{F(a)}, \text{ where } a \text{ is a particular (not arbitrary) constant.}$$

This rule is also called **existential instantiation**:

$$Q_4: \frac{F(a) \text{ for some } a}{(\exists x)F(x)}$$

The rule is called **existential generalisation**

Steps for using Predicate Calculus as a Language for Representing Knowledge

Step 1: Conceptualization: First of all, all the relevant entities and the relations that exist between these entities are explicitly enumerated. Some of the implicit facts, like 'a person dead once is dead forever,' have to be explicated.

Step 2: Nomenclature and Translation: Giving appropriate names to the objects and relations. And then translating the given sentences in English to formulae in FOPL.

Appropriate names are essential in order to guide a reasoning system based on FOPL. It is well-established that no reasoning system is complete. In other words, a reasoning system may need help in arriving at the desired conclusion.

Step 3: Finding an appropriate sequence of reasoning steps, involving the selection of an appropriate rule and appropriate FOPL formulae to which the selected rule is to be applied, to reach the conclusion.

While solving problems with FOPL, generally, the proof technique is proof by contradiction. Under this technique, the negation of what is to be proved is also taken as one of the assumptions. Then from the given assumptions along with the new assumption, we derive a contradiction, i.e., using inference rules, we derive a statement which is the negation of either an assumption or the negation of some earlier derived formula.

Next, we give an example to illustrate how FOPL can be used to solve problems expressed in English and that are solvable by using FOPL. However, the proposed solution does not use the above-mentioned method of contradiction.

We are given the statements:

- (i) *No feeling of pain is publicly observable*
 - (ii) *All chemical processes are publicly observable*
- We are to prove that
- (iii) *No feeling of pain is a chemical process*

Solution:

For translating the given statements (i), (ii) & (iii), let us use the notation:

$F(x)$: x is an instance of feeling of pain
 $O(x)$: x is an entity that is publicly observable
 $C(x)$: x is a chemical process.

Then (i), (ii), and (iii) in FOPL can be equivalently expressed as

- (i) $(\forall x) (F(x) \rightarrow \sim O(x))$
- (ii) $(\forall x) (C(x) \rightarrow O(x))$

To prove

- (iii) $(\forall x) (F(x) \rightarrow \sim C(x))$

From (i) using generalized instantiation, we get

- (iv) $F(a) \rightarrow \sim O(a)$ for any arbitrary a .

Similarly, from (ii), using generalized instantiation, we get

- (v) $C(b) \rightarrow O(b)$ for any arbitrary b .

From (iv), using the transposition rule, we get

- (vi) $O(a) \rightarrow \sim F(a)$ for any arbitrary a

As b is arbitrary in (v), we can rewrite (v) as

- (vii) $C(a) \rightarrow O(a)$ for any arbitrary a

From (vii) and (vi) and using the chain rule, we get

- (viii) $C(a) \rightarrow \sim F(a)$ for any arbitrary a

But as a is arbitrary in (viii), by generalized quantification, we get

- (ix) $(\forall x) (C(x) \rightarrow \sim F(x))$

But (ix) is the same as (iii), which was required to be proved.

Problems with FOPL as a system of knowledge representation and reasoning:

FOPL is not capable of easily representing some kinds of information, including information pieces involving

- (i) properties of relations. For example, the mathematical statement:
Any relation which is symmetric & transitive may not be reflexive
is not expressible in FOPL.
- (ii) linguistic variables like hot, tall, sweat.
For example: *It is very cold today*,
can not be appropriately expressed in FOPL.
- (iii) different belief systems.
For example, *s/he knows that he thinks India will win the match, but I think India will lose*,
also, can not be appropriately expressed in FOPL.

Some Shortcomings of the Predicate Calculus

Now, after having studied predicate logic as a knowledge representation scheme, we ask ourselves the inevitable question of **whether it can be used to solve real-world problems and, if so, to what extent.**

Before we try to answer the above question, let us review some of the properties of logic reasoning systems, including predicate calculus. We must remember that three important properties of any logical reasoning system are **soundness**, **completeness**, and **tractability**. To be confident that an inferred conclusion is true, we require soundness. To be confident that inference will eventually produce any true conclusion, we require completeness. To be confident that inference is feasible, we require tractability.

As discussed above, resolution refutation, as an inference procedure in predicate calculus, is sound and complete. If the well-formed formula to be proved is not logically entailed by the set of well-formed formulas used as premises, the resolution refutation procedure may never terminate. Thus, resolution refutation is not a full decision procedure. Also, there is no other inference procedure with this property that is fully decidable. Therefore, predicate calculus is semi-decidable (i.e., if the set of input well-formed formulas does not entail the conclusion, the procedure may never terminate).

The situation is even more challenging because, for problems on which resolution refutation does terminate, the procedure is NP-hard, as is any sound and complete inference procedure for first-order predicate calculus; that is, it may require exponentially large amounts of time to reach a conclusion.

How to tackle these problems:

Researchers in Artificial Intelligence have proposed various approaches. First, they suggest that we should not insist on the soundness of inference rules. In essence, this means that the rules may occasionally prove a false formula.

Second, they suggest that we should not insist on completeness, thereby allowing the use of procedures that are not guaranteed to prove all true formulas.

Third, they suggest using a language that is less expressive than predicate calculus. For example, a language in which everything is expressed using only Horn Clauses (Horn clauses are those that have at most one positive literal).

9.5 EXAMPLES OF EXPERT SYSTEMS: MYCIN, COMPASS

The first expert system we chose as an example is MYCIN, which is one of the earliest developed expert systems. As another example of an expert system, we briefly discuss COMPASS.

MYCIN (An expert system)

Like everyone else, we are also tempted to discuss MYCIN, one of the earliest designed expert systems at Stanford University in the 1970s.

MYCIN's job was to diagnose and recommend treatment for certain blood infections. To do the proper diagnosis, it is required to grow cultures of the infecting organism, which is a very time-consuming process, and sometimes the patient is in a critical state. So, doctors have to come up with quick guesses about the likely problems from the available data, and use these guesses to provide a treatment where drugs are given, which should deal with any type of problem.

MYCIN was developed to explore how human experts make such preliminary but important judgments based on partial information. Another challenge is that an expert physician may not always be available at every location and at all times. In such situations, an expert system like MYCIN can be extremely useful.

MYCIN represented its knowledge as a set of IF-THEN rules with certainty factors. One of the MYCIN rules could be like :

IF infection is primary-bacteria AND the site of the culture is one of the sterile sites
AND the suspected portal of entry is the gastrointestinal tract
THEN there is suggestive evidence (0.8) that bacteroid infection occurred.

The 0.8 is the certainty that the conclusion will be true given the evidence. If the evidence is uncertain, the certainties of the pieces of evidence will be combined with the certainty of the rule to give the certainty of the conclusion.

MYCIN has been written in Lisp, and its rules are formally represented as Lisp expressions. The action part of the rule could just be a conclusion about the problem being solved, or it could be another Lisp expression.

MYCIN is mainly a **goal-directed system**, using the **backward chaining** reasoning approach. However, to increase its reasoning power and efficiency, MYCIN also uses various heuristics to control the search for a solution.

One of the strategies used by MYCIN is **to first ask the user a number of predefined questions that are most common** and that allow the system to rule out totally unlikely diagnoses. Once these questions have been asked, the system can then focus on particular and more specific possible blood disorders. It then uses a backward chaining approach to try to prove each one. This strategy avoids a lot of unnecessary search and is similar to the way a doctor tries to diagnose a patient.

The other strategies are related to the sequence in which rules are invoked. One of the strategies is simple, i.e., given a possible rule to use, MYCIN first checks all the antecedents of the rule to see if any are known to be false. If yes, then there is no point

using the rule. The other strategies are mainly related to the **certainty factors**. MYCIN first finds the rules that have a greater degree of certainty of conclusions, and abandons its search once the certainties involved get below a minimum threshold, say, 0.2.

There are three main stages to the interaction with MYCIN. In the first stage, initial data about the case is gathered so the system can come up with a broad diagnosis. In the second, more directed questions are asked to test specific hypotheses. At the end of this section, it proposes a diagnosis. In the third stage, it asks questions to determine an appropriate treatment, on the basis of the diagnosis and facts related to the patient. After that, it recommends some treatment. At any stage, the user can ask why a question was asked or how a conclusion was reached, and if a particular treatment is recommended, the user can ask if alternative treatments are possible.

MYCIN has been popular in expert systems research, but it also had a number of problems or shortcomings, because of which a number of its derivatives, like NEOMYCIN, developed.

COMPASS (An expert system)

Before we discuss this, let us understand the functionality of the telephone company's switch. A telephone company's switch is a complex device whose circuitry may encompass a large part of a building. The goals of switch maintenance are to minimize the number of calls that are rerouted due to bad connections and to ensure that faults are repaired quickly. Bad connections are caused by the failure of the connection between two telephone lines.

COMPASS is an expert system that checks error messages derived from the switch's self-test routines, looks for open circuits, reduces the time of operation of components, etc. To find the cause of a switch problem, it looks at a series of such messages and then uses its expertise. The system can suggest the running of additional tests or the replacement of a particular component, for example, a relay or printed circuit board.

As expertise in this area was scarce, it was a fit case for taking help an expert system like COMPASS (We will discuss later, how knowledge acquisition is done in COMPASS).

9.6 EXPERT SYSTEM BUILDING TOOLS

Expert system tools are designed to provide an environment for the development of expert systems, mainly through the approach of prototyping.

The expert systems development process is normally a mixture of prototyping and other incremental development models of software engineering rather than the conventional "waterfall model". Although using incremental development has a problem of integrating new functionality with the earlier version of the expert system, the expert system development environments try to solve this problem by using modular representations of knowledge (some of the representation schemes like frames, rule-based systems, etc. have already been discussed above).

Software tools for the development of expert systems mainly fall into the following categories:

- **Expert System Shells:** These are basically a set of programs, and to be more specific, abstractions over one or more application programs. One of the major examples is EMYCIN, which is the rule interpreter of the famous expert system called MYCIN (a medical diagnostic system). EMYCIN also constitutes related data structures like knowledge tables and an indexing mechanism over these tables. Some recent versions of EMYCIN, like M.4, are very sophisticated shell which combine the backward chaining of EMYCIN with frame – like data structures.
- **High-Level Programming Languages:** These languages are basically used to provide a higher level of abstraction by hiding the implementation details, because of which the programmer does not need to worry about the efficiency of storing, accessing, and manipulating data. One of the good examples is the OPS5 rule language. But many such languages are research tools and may not be available commercially.
- **Multiple programming environments:** These provide a set of software modules to allow the user to use and mix a number of different styles of artificial intelligence programming. One of the examples is called LOOPS, which combines rule-based and object – oriented approaches of knowledge representation.

We will now keep our focus on “Expert System Shells”.

9.6.1 Expert System Shells

An expert system tool, or shell, is a software development environment containing the basic components of expert systems. Associated with a shell is a prescribed method for building applications by configuring and instantiating these components.

Expert system shells are basically used for the purpose of allowing non-programmers to take advantage of the already developed templates or shells, which have evolved because of the efforts of some pioneers in programming who have solved similar problems before. The core components of an expert system are the knowledge base and the reasoning engine.

A generic expert system shell is shown below:

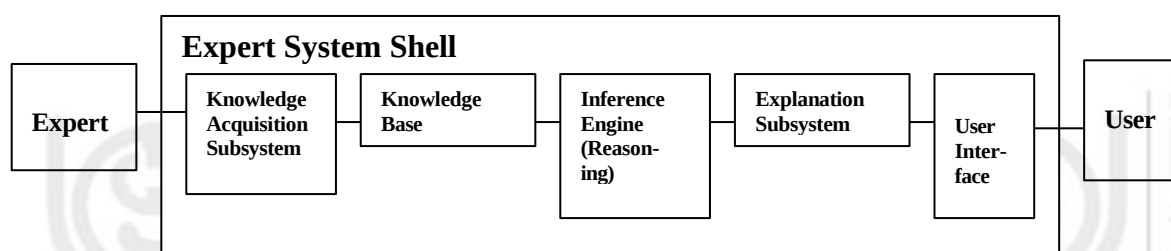


Figure 9.2: Components of Expert System Tool (Shell)

As we can see in the figure above, **the shell includes** the *inference engine*, a *knowledge acquisition subsystem*, an *explanation subsystem*, and a *user interface*. When faced with a new problem in any given domain, we can find a shell that can provide the right support for that problem, so all we need is the knowledge of an expert. There are many commercial expert system shells available now, each one adequate for a different range of problems. Taking the help of expert system shells to develop expert systems greatly reduces the cost and the time of development as compared to developing the expert system from scratch.

Let us now discuss the components of a generic expert system shell. We will discuss:

- **Knowledge Base**
- **Knowledge Acquisition Subsystem**
- **Inference Engine**
- **Explanation Subsystem**
- **User Interface**

9.6.1.1 Knowledge Base

It contains facts and heuristic knowledge. Developers try to use a uniform representation of knowledge as far as possible. There are many knowledge representation schemes for expressing knowledge about the application domain, and some advanced expert system shells use both frames (objects) and IF-THEN rules. In PROLOG, the knowledge is represented as logical statements.

9.6.1.2 Knowledge Acquisition Subsystem

The process of capturing and transforming potentially useful information for a given problem from any knowledge source (which may be a human expert) to a program in the format required by that program is the job of a knowledge acquisition subsystem. So we can say that these subsystems help experts build knowledge bases.

An expert may not be computer-literate, so capturing information includes interviewing, preparing questionnaires, etc., which is a very slow and time-consuming process. So, **collecting knowledge needed to solve problems** and build the knowledge base has always been **the biggest bottleneck** in developing expert systems.

Some of the reasons behind the difficulty in collecting information are given below :

- *The facts and rules or principles of different domains cannot easily be converted into a mathematical or a deterministic model, the properties of which are known.* For example, a teacher knows how to motivate the students, but putting down on paper the exact reasons, causes, and other factors affecting students is not as easy as they vary with individual students.
- *Different domains have their own terminology,* and it is very difficult for experts to communicate their knowledge exactly in normal language.
- *Capturing the facts and principles alone is not sufficient to solve problems.* For example, experts in particular domains whose information is important for specific judgments, such as which information sources are reliable and how problems can be simplified, are based on personal experience. Capturing such knowledge is very difficult.
- *Commonsense knowledge found in humans continues to be very difficult to capture,* although systems are being made to capture it.

The idea of automated knowledge capturing has also been gaining momentum. In fact, **“machine learning”** has been one of the important research areas for some time now. The goal is that a computing system or machine could be enabled to learn in order to solve problems like the way humans do.

Example (Knowledge acquisition in COMPASS).

As we already know, COMPASS is an expert system that was built for proper maintenance and troubleshooting of the telephone company's switches.

Now, for knowledge acquisition, knowledge from a human expert is elicited. An expert explains the problem-solving technique, and a knowledge engineer then converts it into and if-then-rule. The human expert then checks if the rule has the correct logic, and if a change is needed, then the knowledge engineer reformulates the rule.

Sometimes, it is easier to troubleshoot the rules with pencil and paper (i.e., hand simulation), at least the first time, than directly implementing the rule and changing it again and again.

Let us summarize the knowledge acquisition cycle of COMPASS :

- Extract knowledge from a human expert.
- Document the extracted knowledge.
- Test the new knowledge using the following steps:
 - Let the expert analyze the new data.
 - Analyze the same data using pencil and paper using the documented knowledge.
 - Compare the results of the expert's opinion with the conclusion of the analysis using pencil and paper.
 - If there is a difference in the results, then find the rules that are the cause of the discrepancy and then go back to rule 1 to gather more knowledge from the expert to solve the problem.
 - Otherwise, exit the loop.

9.6.1.3 Inference Engine

An inference engine is used to perform reasoning with both the expert knowledge, which is extracted from an expert (most commonly a human expert), and data, which is specific to the problem being solved. Expert knowledge is mostly in the form of a set of IF-THEN rules. The case-specific data includes the data provided by the user and also partial conclusions (along with their certainty factors) based on this data. In a normal forward chaining rule-based system, the case-specific data are the elements in the working memory.

Developing expert systems involves knowing how knowledge is accessed and used during the search for a solution. Knowledge about what is known and when and how to use it is commonly called **meta-knowledge**. In solving problems, a certain level of planning, scheduling, and controlling is required regarding what questions to be asked and when, what is to be checked, and so on.

Different strategies for using domain-specific knowledge have great effects on the performance characteristics of programs, and also on the way in which a program finds or searches for a solution among possible alternatives. Most knowledge representation schemes are used under a variety of reasoning methods, and research is going on in this area.

9.6.1.4 Explanation Subsystem (Example MYCIN)

An explanation subsystem allows the program to explain its reasoning to the user. The explanation can range from how the final or intermediate solutions were arrived at to justifying the need for additional data.

Explanation subsystems are important from the following points of view :

- (i) **Proper use of knowledge:** There must be some assurance for the satisfaction of knowledge engineers that the knowledge is applied properly, even at the time of development of a prototype.
- (ii) **Correctness of conclusions:** Users need to satisfy themselves that the conclusions produced by the system are correct.
- (iii) **Execution trace:** In order to judge that the knowledge elicitation is proceeding smoothly and successfully, a complete trace of program execution is required.
- (iv) **Knowledge of program behavior:** For proper maintenance and debugging, knowledge of program behavior is necessary for the programmers.
- (v) **Suitability of reasoning approach:** Explanation subsystems are necessary to ensure that the reasoning technique applied is suitable for the particular domain.

Explanation in expert systems deals with the issue of control because the reasoning steps used by the programs will depend on how it searches for a solution.

Explanation subsystems are also related to evaluation by checking the outputs produced by a system, and after examining the trace of its actions performed while reasoning, it can be decided whether or not a system is producing the right answer for the right reasons. Without a good explanation of the subsystem, an expert will be unable to judge the system's performance or will be unable to find ways to improve it.

To give an idea of how an explanation subsystem works, let us have a simple example:

Example:

Suppose there is a simple rule-based system **to diagnose the problems in a car.**

Let the system have the following rules :

R1: IF engine-gets-petrol AND engine-turns-over
THEN spark-plug-fault

R2: IF NOT engine-turns-over AND NOT lights-come-on
THEN battery-problem

R3: IF NOT engine-turns-over AND lights-come-on
THEN starter-not-working

R4: IF fuel-tank-has-petrol
THEN engine-gets-petrol

We also assume that the system has no initial facts.

Explanation subsystems allow the user to ask **why** a particular question was asked, and **how** it reached some conclusion. These questions are answered by referring to the system goals, the rules being used, and any existing problem data.

Now let us focus on the example given above to see the explanation facilities provided, which involve a dialogue involving *why* and *how* questions and their explanations.

System: Does the fuel tank has gas?

User: Yes.

System : Does the engine turns over?

User : No

System : Do lights come on?

User : **Why?**

System : **I am trying to prove that there is a battery problem.** It has been established that the engine does not turn over. Therefore, if it is not the case that the lights come on, then there is a battery problem. Is it true that the lights come on?

User : No.

System : I conclude that there is a battery problem.

User : **How?**

System : This follows from Rule 2:

IF NOT engine-turns-over AND NOT lights-come-on was given by the user.

Providing such an explanation facility involves stating what rules are used in reaching conclusions, and using these records to compose explanations like the ones above. Giving simple explanations like those above is not very difficult and is quite useful.

Explanation facilities in expert systems are sometimes not used, and sometimes they are not easily accepted by their users. There are a whole lot of reasons for this. One reason is that the explanations just reference the “surface” knowledge encoded in the rules, rather than providing the “deep” knowledge about the domain, which originally motivated the rules but which is usually not represented. So, the system will say that it concluded something because of rule 5, but not explain what rule 5 intends to say. In the example given below, maybe the user needs to understand that both the lights and the starter use the battery, which is the underlying purpose of the second rule in this example. Another reason for the frequent failure of explanation facilities is the fact that, if the user fails to understand or accept the explanation, the system can’t re-explain in another way (as people can). Explanation generation is a fairly large area of research, concerned with effective communication, i.e., how to present things so that people are really satisfied with the explanation, and what implications does this have for how we represent the underlying knowledge.

Explanation Subsystem in MYCIN (An overview)

MYCIN is one of the first popular expert systems made for the purpose of medical diagnosis. Let us have a look at how the explanation subsystem in MYCIN works: To explain the reasoning for deciding on a particular medical parameter’s or symptom’s value, it retrieves a set of rules and their conclusions. It allows the user to ask questions or queries during a consultation.

To answer the questions, the system relies on its ability to display a rule invoked at any point during the consultation, and also recording the order of rule invocations and associating them with particular events (like particular questions).

As the system uses backward chaining, most of the questions belong to the “Why” or “How” category. To answer “Why” questions, the system looks up the hierarchy (i.e., tree) of rules to see which goals the system is trying to achieve and to answer “Why.”

questions, the system must look down the hierarchy (i.e., tree) to find out which sub-goals were satisfied to achieve the goal.

We can see that the explanation process is nothing but a search problem requiring tree traversal.

As MYCIN keeps track of the goal-to-sub-goal sequence of the computation, so it can answer questions like:

“Why did you ask that if the stain of the organism is gram-negative?”

In response to this, the system would quote the rules which state “gram negative staining” may be in conjunction with other conditions, and would state what it was trying to achieve.

Similarly, if there is “How” question like:

“How do say that Organism-2 might be proteus?”

In its reply, MYCIN would state the rules that were applied in reaching these conclusions, their degree of certainty, and what was the last question asked etc.

Thus, we can see that because of the backward chaining approach, the system is able to answer “Why” and “How” questions satisfactorily. But the rule application would not be easy if the chains of reasoning are long.

9.6.1.5 User interface

It is used to communicate with the user. The user interface is generally not a part of the expert system technology, and was not given much attention in the past. However, it is now widely accepted that the user interface can make a critical difference in the utility of a system regardless of the system’s performance.

Now, as an example, let us discuss an expert system shell called EMYCIN.

9.6.1.6 EMYCIN (An expert system shell)

EMYCIN provides a domain-independent framework or template for constructing and running any consultation programs. EMYCIN stands for “Empty MYCIN” or “Essential MYCIN” because it basically constitutes a MYCIN system minus its domain-specific medical knowledge. However, EMYCIN is something more than this, as it offers a number of software tools for helping expert system designers in building and debugging performance programs.

Some characteristics of EMYCIN are:

- It constitutes an abbreviated rule language, which uses ALGOL-like notation and which is easier than LISP and is more concise than the English subset used by MYCIN.
- It uses backward chaining, which is similar to MYCIN.
- It indexes rules, in turn organising them into groups, based on the parameters that are being referenced.
- It has an interface for system designers that provides tools for displaying, editing, and partitioning rules, editing knowledge held in tables, and also running rule sets on sets of problems. As part of the system designer’s interface, EMYCIN also included a knowledge editor (a program) called TEIRESIAS whose job was to provide help for the development and maintenance of large knowledge bases.

- It also has a user interface that allows the user to communicate with the system smoothly.

9.7 SOME APPLICATION AREAS OF EXPERT SYSTEMS

The scope of applications of expert systems technology to practical problems is so wide that it is very difficult to characterize them. The applications find their way into most of the areas of knowledge work. Some of the main categories of applications or an expert system are given below.

- **Diagnosis and Troubleshooting:** This class comprises systems that deduce faults and suggest corrective actions for a malfunctioning device or process. Medical diagnosis was one of the first knowledge areas to which ES technology was applied, but the use of expert systems for solving and diagnosing engineered systems has become common.
- **Planning and Scheduling:** Systems that fall into this class analyze a set of one or more potentially complex and interacting goals in order to determine a set of actions to achieve those goals. This class of expert systems has great commercial potential. Examples include scheduling of flights, personnel, manufacturing process planning, etc.
- **Process Monitoring and Control:** Systems falling in this class analyze real-time data from physical devices to notice errors, predict trends, and control for both optimality and failure correction. Examples of real-time systems that actively monitor processes are found in the steel-making and oil refining industries.
- **Financial Decision Making:** The financial services industry has also been using expert system techniques. Expert systems belonging to this category act as advisors, risk analyzers, etc.
- **Knowledge Publishing:** This is a relatively new, but also potentially explosive area. The primary function of the expert system is to deliver knowledge that is relevant to the user's problem, in the context of the user's problem.
- **Design and Manufacturing:** These systems assist in the design of physical devices and processes, ranging from high-level conceptual design of abstract entities all the way to factory floor configuration of manufacturing processes.

9.8 SUMMARY

In this unit, we have discussed various issues in respect of expert systems. To begin with, in section 1.2, we define the concept of 'expert system'. In view of the fact that an expert system contains knowledge of a specific domain, in section 1.4, we discuss various schemes for representing the knowledge of a domain. Section 1.5 contains examples of some well-known expert systems. Tools for building expert systems are discussed in Section 1.6. Finally, applications of expert systems are explained in Section 1.7.

9.9 SOLUTIONS/ANSWERS

Ex. 1) In order to translate in PL, let us use the symbols:

ML: There is a Moral Law

SG: Someone Gave it (the word 'it' stands for moral law)

TG: There is God.

Using these symbols, the **statements** (i) to (iv) become the **formulae** (i) to (iv) of PL as given below:

(i) ML

(ii) $ML \rightarrow SG$

(iii) $SG \rightarrow TG$ and

(iv) TG

Applying Modus Ponens to formulae (i) and (ii), we get the formula

(v) SG

Applying Modus Ponens to (v) and (iii), we get

(vi) TG

But formula (vi) is the same as (iv), which is required to be established. Hence the proof.

9.10 FURTHER READINGS

1. Russell S. & Norvig P., *Artificial Intelligence A Modern Approach* (Second Edition) (Pearson Education, 2003).
2. Patterson D. W: *Introduction to Artificial Intelligence and Expert Systems* (Prentice Hall of India, 2001).

